Data Preparation

This document explains the **data preprocessing stage** for the Customer Churn Analysis project.

The goal of this stage is to prepare the raw telecommunications dataset for machine learning and clustering analysis. Proper data preparation improves the quality of the analysis and ensures that the dataset is suitable for modeling.

The preprocessing process includes:

- Loading and inspecting the dataset
- Identifying and handling missing values
- Removing duplicate records
- Encoding categorical variables
- Converting the target variable into a numeric format
- Producing a fully cleaned and structured dataset

The resulting dataset will be saved as a **preprocessed dataset** and used in later stages of the project such as **training/testing split, scaling, clustering analysis, and predictive modeling**.

Removing Duplicate Records

Duplicate records can lead to biased analysis and distort clustering results. Therefore, it is important to check for duplicate entries within the dataset.

If duplicate rows are identified, they were removed to ensure that each record represents a unique customer.

```
Number of duplicate rows: 302
Shape after removing duplicates: (6741, 10)
```

Encoding Target Variable

The target variable **Churn** indicates whether a customer has left the service provider.

The values are originally represented as text:

- Yes
- No

To enable machine learning algorithms to process this variable, it was converted into a binary numerical format:

- **Yes → 1 (Customer churned)**
- **No → 0 (Customer retained)**

This transformation allowed churn prediction models to treat the problem as a binary classification task.

```
df_encoded['Churn'] = df['Churn'].map({'Yes':1,'No':0})
print("Unique values in Churn column:", df_encoded['Churn'].unique())
```

```
Unique values in Churn column: [1 0]
```

Purpose of Splitting the Dataset

Before training machine learning models, the dataset must be divided into **separate training and testing sets**. This process allows the model to learn patterns from one portion of the data and then evaluate its performance on unseen data.

Splitting the dataset helps to:

- Prevent **overfitting**, where the model memorizes training data instead of learning general patterns.
- Provide an **unbiased evaluation** of model performance.
- Ensure that the trained model can **generalize to new customers**.

In this project, the dataset was split using the **train_test_split function from the scikit-learn library**.

Method Used for Splitting the Dataset

The dataset was divided into two subsets:

- **Training set:** Used to train machine learning models.
- **Testing set:** Used to evaluate model performance.

The following parameters were used for the split:

Parameter	Value	Purpose
test_size	0.2	20% of data used for testing
random_state	42	Ensures reproducibility
stratify	y	Maintains churn distribution

Stratification ensures that both training and testing sets maintain **the same proportion of churned and non-churned customers** as the original dataset.

Step-by-Step Process

Step 1: Separate Features and Target Variable

The dataset was divided into:

- **Feature variables (X)** — input variables used for prediction.
- **Target variable (y)** — the churn column representing whether a customer churned.

Feature variables include:

- SeniorCitizen
- tenure
- MonthlyCharges
- gender_Male
- Dependents_Yes
- PhoneService_Yes
- MultipleLines_Yes
- InternetService_Fiber optic
- Contract_One year
- Contract_Two year
- Churn_Yes

```
Features (X) shape: (6741, 11)
Target (y) shape: (6741,)
Feature columns:
 ['SeniorCitizen', 'tenure', 'MonthlyCharges', 'gender_Male', 'Dependents_Yes', 'PhoneService_Yes', 'MultipleLines_Yes', 'InternetService_Fiber optic', 'Contract_One year',
```

Perform Train/Test Split

The dataset was split using the **train_test_split function** with an 80/20 ratio.

- **80% of the dataset** was used for training.
- **20% of the dataset** was reserved for testing.

This ensures that the model is evaluated on **unseen data**, which provides a realistic estimate of its predictive performance.

Verify Target Distribution

After splitting the dataset, it is important to verify that the **distribution of the target variable remains consistent** between the training and testing sets.

This ensures that the model is trained and evaluated on **representative samples**.

```
Training features shape: (5392, 11)
Testing features shape: (1349, 11)
Training target shape: (5392,)
Testing target shape: (1349,)

Churn distribution in training set:
 Churn
0    0.734236
1    0.265764
Name: proportion, dtype: float64

Churn distribution in testing set:
 Churn
0    0.734618
1    0.265382
Name: proportion, dtype: float64
```

Saving the Train and Test Datasets

After splitting, the datasets were saved as separate files to ensure **reproducibility and proper project organization**.

```python
# Save features
X_train.to_csv("c:/Users/Prometheus/.vscode/Customer_Churn_Analysis/X_train.csv", index=False)
X_test.to_csv("c:/Users/Prometheus/.vscode/Customer_Churn_Analysis/X_test.csv", index=False)

# Save targets
y_train.to_csv("c:/Users/Prometheus/.vscode/Customer_Churn_Analysis/y_train.csv", index=False)
y_test.to_csv("c:/Users/Prometheus/.vscode/Customer_Churn_Analysis/y_test.csv", index=False)

print("Training and testing sets saved successfully in Data_Preparation folder!")
```

```
Training and testing sets saved successfully in Data_Preparation folder!
```

Inspect Feature Statistics Before Scaling

Before applying scaling, the statistical distribution of features in the training dataset was examined using the `.describe()` function. This step helped identify the differences in magnitude across variables.

```
Feature statistics BEFORE scaling:

       SeniorCitizen       tenure  MonthlyCharges
count    5392.000000  5392.000000     5392.000000
mean        0.168769    33.029859       65.959013
std         0.374582    24.315112       29.636016
min         0.000000     0.000000       18.000000
25%         0.000000    10.000000       42.000000
50%         0.000000    30.000000       71.000000
75%         0.000000    56.000000       90.000000
max         1.000000    72.000000      119.000000
```

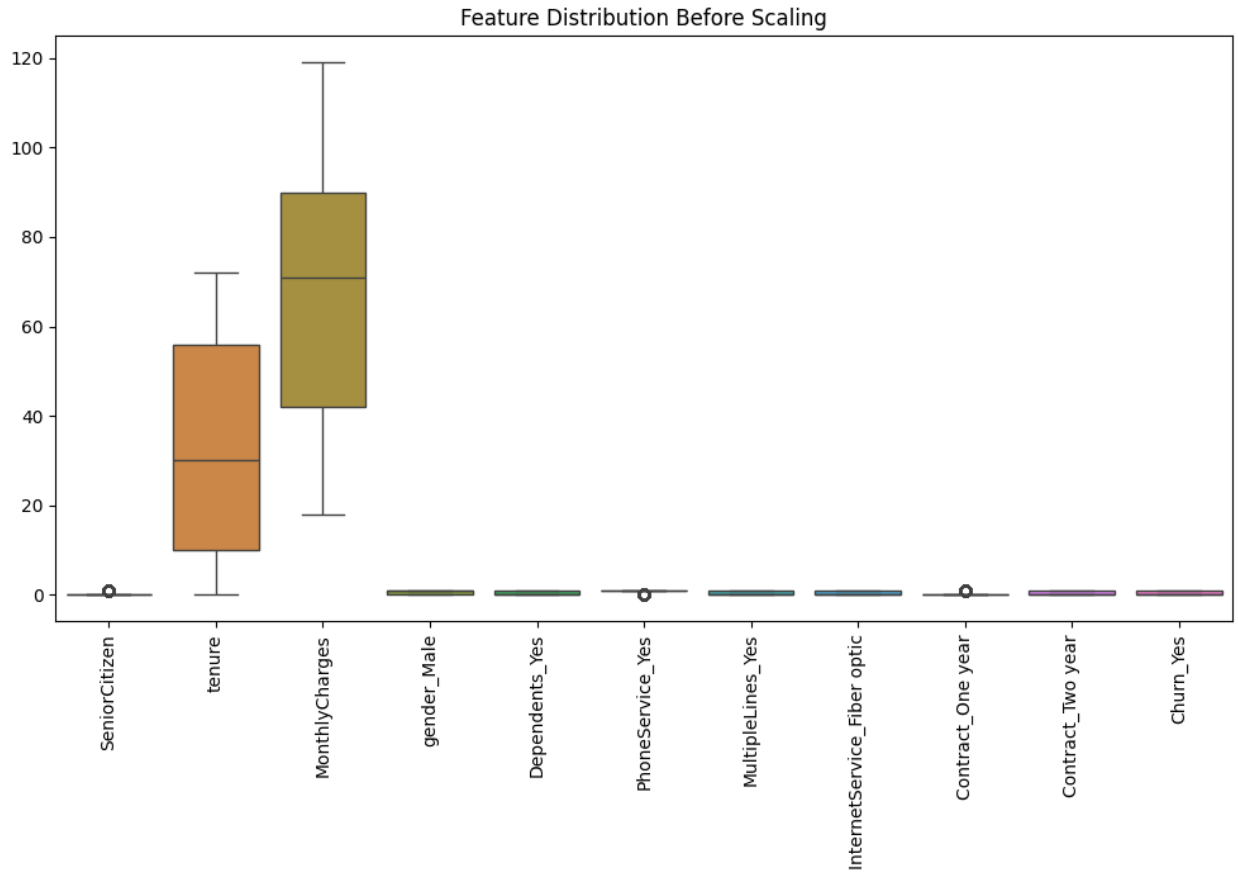
Visualize Feature Distribution Before Scaling

To better understand the scale differences between variables, a **boxplot visualization** was generated.

This visualization highlights:

- Range differences

- Outliers
- Feature distribution



Initialize the StandardScaler

The StandardScaler object from the **scikit-learn preprocessing module** was initialized.

The scaler calculates the **mean and standard deviation** from the training dataset.

Important:
The scaler is **fitted only on the training data** to avoid data leakage.

Fit the Scaler on the Training Dataset

The scaler was fitted using the training dataset to compute scaling parameters.

```
Scaler fitted to training data.
Feature means learned by scaler:
 [ 0.16876855 33.02985905 65.95901335  0.50667656  0.30433976  0.89706973
  0.4245178   0.43916914  0.21383531  0.25259644  0.26576409]
Feature standard deviations learned by scaler:
 [ 0.37454736 24.31285711 29.63326767  0.49995542  0.46012723  0.30386778
  0.4942696   0.49628581  0.41001192  0.43450141  0.44173922]
```

Apply Scaling Transformation

After fitting the scaler, the transformation was applied to both:

- **Training dataset**
- **Testing dataset**

This ensures both datasets are normalized using the same parameters.

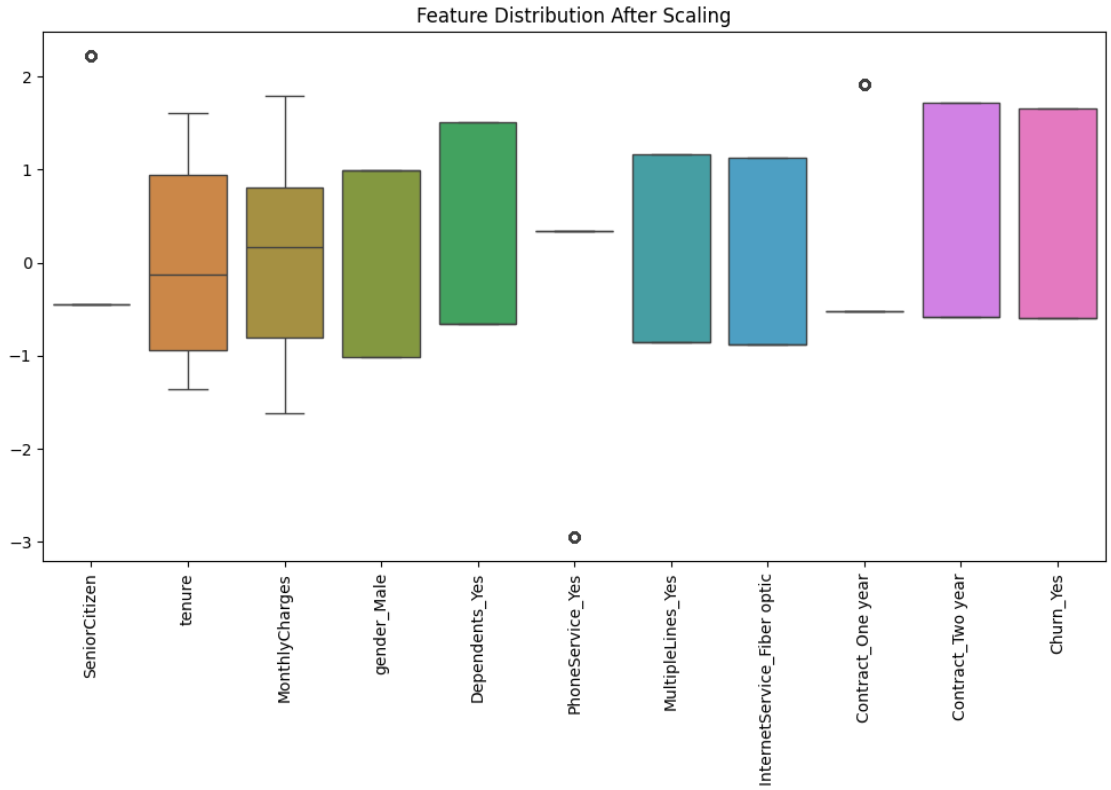
```
Scaling applied successfully.
Scaled training dataset shape: (5392, 11)
Scaled testing dataset shape: (1349, 11)
```

Verify Scaling Results

To confirm the transformation worked correctly, the statistical properties of the scaled dataset were examined.

Visualize Feature Distribution After Scaling

A second boxplot visualization was generated to compare feature distributions after scaling.



Result of Feature Scaling

After normalization:

- All features have comparable scales.
- Bias caused by large-value variables has been removed.
- The dataset is now suitable for machine learning algorithms such as:
    - **K-Means clustering**
    - **Artificial Neural Networks**

The scaled datasets were saved for further analysis.